Introduction

Graph

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

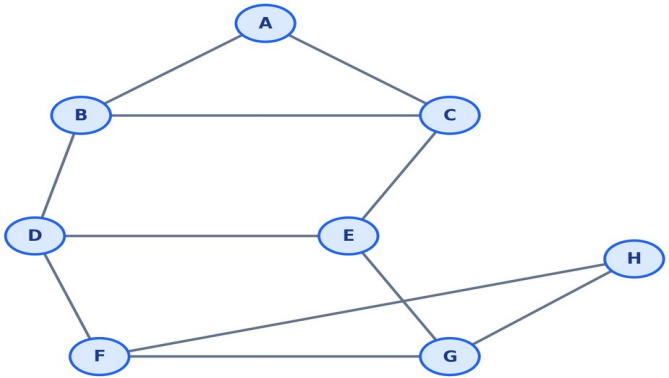
Graph

Definition

A **graph** is a **non-linear data structure** used to represent relationships between objects. It consists of vertices (nodes) and edges connecting pairs of vertices.

G = (V, E)

V is the set of vertices and E is the set of edges.



There are 8 vertices or nodes.

V = { A, B, C, D, E, F, G, H}

There are 11 edges.

AB, AC, BC, BD, CE, DE, DF, FG, EG, GH, FH

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Terminology

Vertex (node): An object represented in the graph, such as A.
Edge: A connection between two vertices, such as AB.
Adjacent vertices: Vertices connected directly by an edge; A and B are adjacent.
Incident edge: An edge is incident on each endpoint; AB is incident on A and B.
Degree: The number of edges incident on a vertex in an undirected graph.
In-degree: The number of directed edges entering a vertex.
Out-degree: The number of directed edges leaving a vertex.
Path: A sequence of vertices joined by edges, such as A–B–D–F–H.
Path length: The number of edges in a path; A–B–D–F–H has length 4.

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Terminology

Cycle: A path returning to its starting vertex without repeating intermediate vertices, such as A–B–C–A.
Connected graph: An undirected graph where every vertex is reachable from every other vertex.
Connected component: A maximal connected part of a graph.
Simple graph: A graph without self-loops or multiple edges between the same pair of vertices.
Self-loop: An edge connecting a vertex to itself.
Complete graph: A simple undirected graph where every pair of distinct vertices is connected.
Subgraph: A graph formed from a subset of another graph's vertices and edges.
Cyclic / acyclic graph: A graph that contains / does not contain a cycle.

© Confidential Information of PES University. All Rights Reserved - 2026

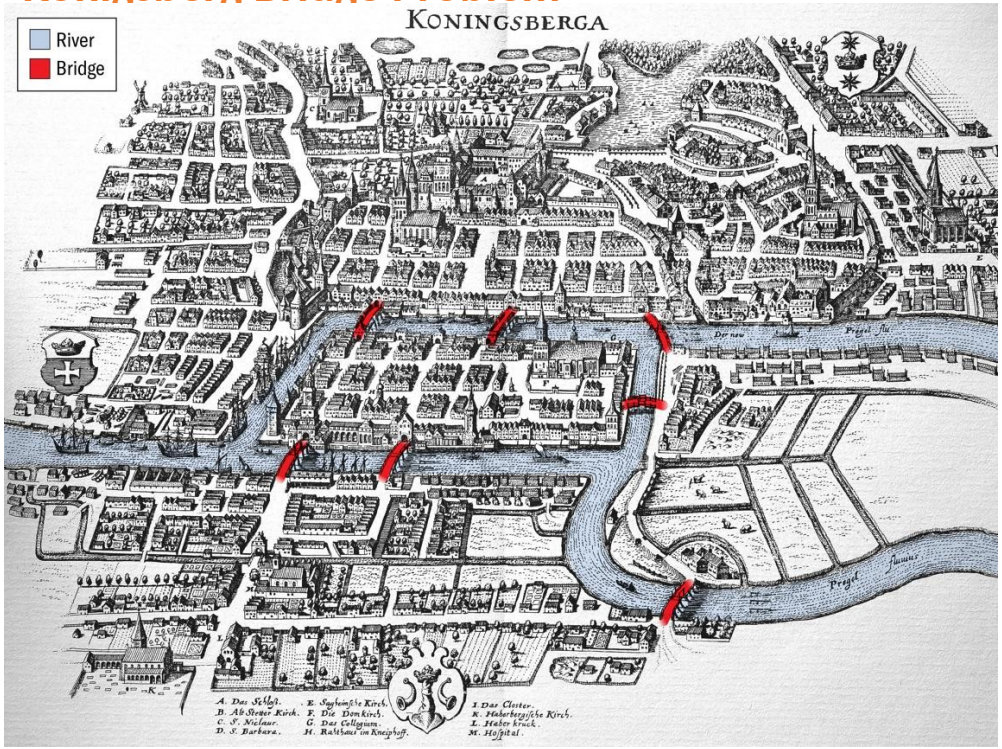
example

Number of vertices: |V| = 8.
Number of edges: |E| = 11.
Neighbours of A: B and C.
Degree of A: deg(A) = 2.
Degree of B: deg(B) = 3 (edges AB, BC and BD).
One path from A to H is A–B–D–F–H; its length is 4.
A cycle is A–B–C–A.
The graph is connected because every vertex can be reached from every other vertex.

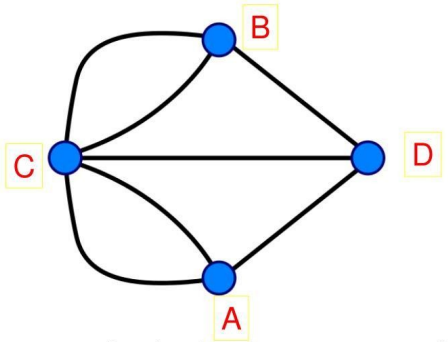
© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Königsberg Bridge Problem



The seven bridges of Konigsberg =



Can you start and end at the same vertex, traversing every edge only once?

© Confidential Information of PES University. All Rights Reserved - 2026

Euler graph and Hamilton graph

A graph is said to be Eulerian if it supports the traversal of all edges once and only once and come back to the starting point. The necessary and sufficient condition is that the degree of each node should be even.

A graph is said to be Hamiltonian if it supports the traversal all vertices once and only once and come back the starting point. We do not have an efficient solution for this.

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Applications:

Road and transport networks: Vertices are cities or intersections; edges are roads. Weights can represent distance or travel time.
Social networks: Vertices are people; edges are friendships, follows or interactions.
Computer networks: Vertices are computers or routers; edges are communication links.
Course prerequisites: A directed edge can mean one course must be completed before another.

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

Applications:

Software dependencies: Vertices are modules, files or tasks; edges represent dependencies.
Airline and logistics networks: Vertices are airports or warehouses; edges represent routes.
Web pages: Vertices are pages and directed edges represent hyperlinks.
Project scheduling: Vertices represent tasks and edges represent precedence constraints

© Confidential Information of PES University. All Rights Reserved - 2026

Graph

ADT:

Operation	Purpose
create_graph()	Create an empty graph.
add_vertex(v)	Insert a vertex.
remove_vertex(v)	Remove a vertex and its incident edges.
add_edge(u, v)	Add an edge between u and v.
remove_edge(u, v)	Remove an edge.
has_vertex(v)	Check whether v exists.
has_edge(u, v)	Check whether an edge exists.
get_neighbors(v)	Return vertices adjacent to v.
degree(v)	Return the degree of v in an undirected graph.
display_graph()	Display vertices and edges.
destroy_graph()	Release resources used by the graph.

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)